

Neural Networks and the Chomsky Hierarchy

Empirical Limits of Neural Generalization & Curricular Relevance to CS 305

Author Analysis & Pedagogical Integration for Theory of Computation (CS 305)

Ghana Institute of Management and Public Administration (GIMPA) | School of Technology
Level 300 Bachelor of Science in Computer Science | Academic Year 2026/2027

Executive Summary & Primary Citation

Paper Citation: Delétang, G., Ruoss, A., Grau-Moya, J., Genewein, T., Wenliang, L. K., Catt, E., Cundy, C., Hutter, M., Legg, S., Veness, J., & Ortega, P. A. (2023). *Neural Networks and the Chomsky Hierarchy*. International Conference on Learning Representations (ICLR 2023). DeepMind & University of Oxford.

Core Finding: Neural network architectures do not generalize uniformly. By evaluating **20,910 trained models** across **15 algorithmic transduction tasks**, DeepMind proved empirically that **an architecture's out-of-distribution (OOD) length generalization is fundamentally bounded by its formal computational class on the Chomsky Hierarchy**. Recurrent Neural Networks (RNNs) and standard Transformers fail on non-regular tasks; Long Short-Term Memory (LSTM) networks succeed only up to counter languages; only architectures explicitly augmented with structured external memory (Stack-RNNs for Context-Free tasks, Tape-RNNs for Context-Sensitive tasks) successfully generalize to longer algorithmic sequences.

1. The Research Motivation: The Central Paradox of Deep Learning

In contemporary machine learning, the dominant paradigm for measuring generalization is **Statistical Learning Theory** (Vapnik, 1998), which assumes that training and test samples are drawn independently and identically distributed (*i.i.d.*) from the same underlying distribution $\mathcal{D}$:

$$(x_{train}, y_{train}) \sim \mathcal{D}, \quad (x_{test}, y_{test}) \sim \mathcal{D}. \quad (1)$$

Under the *i.i.d.* assumption, modern Deep Neural Networks (DNNs) achieve state-of-the-art results on NLP, computer vision, and code synthesis. However, real-world robust AI requires **Out-of-Distribution (OOD) Generalization** and **Algorithmic Extrapolation**—specifically, learning an underlying generative rule on small instances (e.g., strings of length $n \leq 40$) and executing that exact procedure faithfully on arbitrarily long instances ($n > 40$, up to $n = 500$ or ∞).

Theoretical Turing-Completeness vs. Practical Realizability

A major source of confusion in computer science literature is the gap between asymptotic expressivity and practical learnability:

- **Theoretical Universal Expressivity:** Classical theorems establish that Recurrent Neural Networks with infinite precision weights (Siegelmann & Sontag, 1994) and standard Transformers with unbounded precision or dynamic length (Pérez et al., 2019, 2021) are **Turing-complete**.
- **The Practical Reality:** Real-world models operate with finite float precision, finite compute budgets, and are optimized via stochastic gradient descent (Adam/SGD). When trained on algorithmic sequence tasks, standard models fail catastrophically as soon as the sequence length exceeds the training envelope.

DeepMind's landmark paper replaces statistical heuristics with **Formal Language Theory**, asking: *Can the 70-year-old Chomsky Hierarchy serve as a rigorous predictive framework for the empirical generalization boundaries of modern neural networks?*

2. Theoretical Framework: Languages, Automata, and Transduction

The Chomsky Hierarchy of Formal Languages

In formal language theory (Chomsky, 1956; Sipser, 2013), an alphabet Σ is a non-empty finite set of symbols, and a language $L \subseteq \Sigma^*$ is a set of finite-length strings. The Chomsky Hierarchy classifies grammars and their recognized languages according to the production rules and the corresponding minimal computational automaton required:

Grammar Level	Minimal Automaton	Memory Structure	Production Constraint
Regular (L_3)	Finite-State Automaton (DFA/NFA)	Internal state ($O(1)$ memory)	$A \rightarrow a$ or $A \rightarrow aB$
Context-Free (L_2)	Pushdown Automaton (PDA)	Unbounded LIFO Stack (top access)	$A \rightarrow \alpha$
Context-Sensitive (L_1)	Linear Bounded Automaton (LBA)	Bounded Tape (size $O(n)$)	$\alpha A \beta \rightarrow \alpha \gamma \beta$ ($ \gamma \geq A $)
Recursively Enumerable (L_0)	Turing Machine (TM)	Unbounded Read/Write Tape	$\gamma \rightarrow \alpha$ ($\gamma \neq \epsilon$)

Table 1: The Chomsky Hierarchy, associated automata, and memory models.

Formulation: From Language Recognition to Language Transduction

Classical theory defines formal languages via **decision problems** (recognition):

$$f_{\text{rec}} : \Sigma^* \rightarrow \{0, 1\} \quad \text{where } f_{\text{rec}}(w) = 1 \iff w \in L. \quad (2)$$

However, modern sequence-to-sequence neural networks (such as Transformers and RNNs) perform **language transduction**, mapping an input string to an output string:

$$f_{\text{trans}} : \Sigma_I^* \rightarrow \Sigma_O^* \quad \text{where } y = f_{\text{trans}}(x). \quad (3)$$

Importantly, the computational automata required for language transduction are identical to those for recognition, but equipped with output emission transitions (e.g., Mealy/Moore machines for regular languages, pushdown transducers for context-free languages, and Turing transducers for general computation).

Experimental Length-Generalization Protocol

To evaluate genuine algorithmic learning rather than rote memorization:

- Training Distribution:** Strings sampled with length $\ell \sim \mathcal{U}(1, N)$ where $N = 40$.
- Out-of-Distribution Test Distribution:** Strings sampled with length $\ell \sim \mathcal{U}(N + 1, M)$ where $M = 500$ (over $12.5\times$ longer than the maximum training length).
- Success Criterion:** An architecture is considered to successfully generalize if its test accuracy across the entire out-of-distribution range $[41, 500]$ satisfies:

$$\text{Score} = \frac{1}{M - N} \sum_{\ell=N+1}^M \text{Acc}(\ell) \geq 90.0\%. \quad (4)$$

3. Neural Network Architectures & Memory Augmentation

The DeepMind benchmark systematically compared five foundational architectural families:

1. **Vanilla Recurrent Neural Network (RNN)**: Employs a dense hidden vector $\mathbf{h}_t = \tanh(W\mathbf{x}_t + U\mathbf{h}_{t-1} + \mathbf{b})$. The hidden state $\mathbf{h}_t$ must simultaneously compress past history and serve as computational working memory. Because $\mathbf{h}_t \in \mathbb{R}^d$ has fixed dimension, it corresponds computationally to a continuous **Finite-State Machine**.
2. **Long Short-Term Memory (LSTM)**: Introduces additive cell state updates $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$ regulated by input, forget, and output gates. The linear cell update avoids vanishing gradients and allows neurons to behave as **continuous counters**, theoretically enabling recognition of Counter Languages (a subset of Deterministic Context-Free Languages).
3. **Stack-RNN**: An RNN controller augmented with a differentiable continuous stack memory (Joulin & Mikolov, 2015). At each step, the controller emits soft action probabilities for {PUSH, POP, NO_OP} and a content vector $\mathbf{v}_t$. The memory behaves as a LIFO buffer, mimicking a **Pushdown Automaton (PDA)**.
4. **Tape-RNN**: An RNN controller augmented with an external random-access read/write tape and movable heads (inspired by Neural Turing Machines; Graves et al., 2014, 2016). The head can execute {Write-Stay, Write-Left, Write-Right, Jump-Left, Jump-Right}. This explicitly models a **Linear Bounded Automaton (LBA)** or **Turing Machine (TM)**.
5. **Transformer (Self-Attention)**: Multi-head scaled dot-product attention network:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (5)$$

Evaluated with five positional encoding schemes: No Positional Encoding, Standard Sinusoidal, ALiBi (Attention with Linear Biases), RoPE (Rotary Position Embeddings), and Transformer-XL recurrence.

Level	Transduction Task	RNN	LSTM	Transf.	Stack-RNN	Tape-RNN
Regular Languages (L_3) — Minimal Automaton: Finite-State Automaton (DFA/NFA)						
	Even Pairs	100.0	100.0	96.4	100.0	100.0
	Modular Arithmetic (Simple)	100.0	100.0	24.2	100.0	100.0
	Parity Check [†]	100.0	100.0	52.0	100.0	100.0
	Cycle Navigation [†]	100.0	100.0	61.9	100.0	100.0
Deterministic Context-Free Languages (L_2) — Minimal Automaton: Pushdown Automaton (PDA)						
	Stack Manipulation	56.0	59.1	57.5	100.0	100.0
	Reverse String (w^R)	62.0	60.9	62.3	100.0	100.0
	Modular Arithmetic (Nested)	41.3	59.2	32.5	96.1	95.4
	Solve Equation [°]	51.0	67.8	25.7	56.2	64.4
Context-Sensitive Languages (L_1) — Minimal Automaton: Linear Bounded Automaton (LBA)						
	Duplicate String (ww)	50.3	57.6	52.8	52.8	100.0
	Missing Duplicate	52.3	54.3	56.4	55.2	100.0
	Odds First	51.0	55.6	52.8	51.9	100.0
	Binary Addition	50.3	55.5	54.3	52.7	100.0
	Binary Multiplication [×]	50.0	53.1	52.2	52.7	58.5
	Compute Sqrt	54.3	57.5	52.4	56.5	57.8
	Bucket Sort ^{†*}	27.9	99.3	91.9	78.1	70.7

Table 2: OOD length generalization score ($\geq 90.0\%$ in **bold**). Random guessing baseline is 50.0% (20.0% for modular tasks). Annotations: [†]Permutation-invariant; ^{*}Counting task; [°]Requires non-deterministic search; [×]Requires superlinear computational time.

4. Main Findings and Empirical Results

Table 2 summarizes the benchmark generalization scores across 20,910 trained models evaluated on the 15 algorithmic tasks.

Key Empirical Insights from the Results

1. Exact Architectural Hierarchy Alignment:

- **RNN $\approx$ Finite Automaton (L_3):** Successfully masters all Regular language tasks (100.0% on Even Pairs, Parity, Modular Simple, Cycle Navigation) but drops immediately to near-chance performance on Context-Free tasks (e.g., 56.0% on Stack Manipulation, 62.0% on Reverse String).
- **Stack-RNN $\approx$ Pushdown Automaton (L_2):** Perfect 100.0% generalization on deterministic context-free tasks requiring nested LIFO storage (Reverse String, Stack Manipulation, Nested Arithmetic), but completely fails on Context-Sensitive tasks requiring cross-serial dependencies (*ww* Duplicate String: 52.8%).
- **Tape-RNN $\approx$ Linear Bounded Automaton (L_1):** The *only* architecture that generalizes on Context-Sensitive tasks (*ww* Duplicate String: 100.0%, Missing Duplicate: 100.0%, Odds First: 100.0%, Binary Addition: 100.0%).

2. The LSTM Counter-Language Anomaly:

LSTMs surpass standard RNNs on certain tasks because their linear cell state $\mathbf{c}_t$ acts as an unbounded numerical accumulator. Consequently, LSTMs solve **Counter Languages** (e.g., Bucket Sort at 99.3%, which requires tracking token counts). However, because a scalar counter cannot preserve sequence order, LSTMs fail on Reverse String (60.9%) and general context-free parsing.

3. The Failure Modes of Standard Transformers:

Despite powering modern Large Language Models (GPT-4, Claude, Gemini), standard Transformers exhibited severe algorithmic limitations:

- **Failure on Regular Permutation-Invariant Tasks:** Transformers failed on Parity Check (52.0%, essentially random coin-flip). Soft-max attention over large sequence lengths dilutes attention weights ($\frac{1}{n}$ diffusion), and positional encodings break permutation invariance.
- **Out-of-Distribution Positional Encoding Breakdown:** When sequence lengths exceed the training window ($n > 40$), positional embeddings (whether Absolute, RoPE, or ALiBi) produce representation values never seen during training, driving first-layer activations out-of-distribution (confirmed via PCA in Figure 4 of the paper).

4. Computational Complexity Bottlenecks (P vs. Real-Time Bounds):

Tape-RNNs failed on **Binary Multiplication** (58.5%) and **Compute Square Root** (57.8%). This is not because LBAs cannot represent multiplication, but because standard recurrent models are constrained to $O(n)$ time steps. Multi-digit binary multiplication fundamentally requires $\Omega(n^2)$ bit operations (or superlinear time). Constraining the neural network to a single pass without extra computational time steps makes length generalization mathematically impossible.

5. Mechanistic Interpretability: What Did the Networks Learn?

A profound contribution of the DeepMind study was probing the internal learned representations of successful models to determine whether they discovered genuine classical automata algorithms:

- **RNN on Parity Check (DFA Reconstruction):** Dimensionality reduction (PCA) of the RNN hidden states $\mathbf{h}_t$ revealed that the continuous vector space self-organized into **two distinct topological attractor clusters**, corresponding exactly to state q_{even} and state q_{odd} in the canonical 2-state Parity DFA.
- **Stack-RNN on Nested Modular Arithmetic (PDA Reconstruction):** Analysis of the Stack-RNN's continuous memory operations revealed that the network autonomously learned the textbook

parsing algorithm:

$$\sigma(x_t) = \begin{cases} \text{PUSH} & \text{if } x_t = '(' \text{ (open bracket, saving accumulator),} \\ \text{NO_OP} & \text{if } x_t \in \{0, 1, 2, 3, 4, +, -, *\} \text{ (internal controller arithmetic),} \\ \text{POP} & \text{if } x_t = ')' \text{ (close bracket, retrieving outer context).} \end{cases} \quad (6)$$

- **Tape-RNN on Duplicate String (Turing Machine Head Trajectory):** Inspection of the Tape-RNN's read/write heads on $w \rightarrow ww$ showed discrete phases: (i) sequential storage of w using WRITE-LEFT, (ii) head relocation via JUMP-RIGHT, and (iii) emission of duplicate tokens while tracking tape boundaries.

6. Curricular Alignment: Integrating DeepMind's Findings into CS 305

The DeepMind paper provides a gold standard bridge between theoretical automata and 21st-century Artificial Intelligence. Below is the direct pedagogical mapping to the four core modules of GIMPA's **CS 305: Theory of Computation**:

Module-by-Module Integration for CS 305

Module 1: Regular Languages & Finite Automata (Weeks 1–4) • Course Concept: DFAs, NFAs, state minimization, Pumping Lemma for Regular Languages.

- **DeepMind Connection:** The paper proves that Vanilla RNNs are the exact neural analogue of DFAs. The hidden state vector $\mathbf{h}_t$ represents the state transition function $\delta(q, a)$.
- **Lecture Case Study:** Demonstrate how Parity Check ($L = \{w \in \{0, 1\}^* \mid |w|_1 \text{ is even}\}$) requires exactly 2 states in a DFA. Show why Transformers fail this simple regular task without recurrence, while RNNs achieve 100% generalization.

Module 2: Context-Free Languages & Pushdown Automata (Weeks 5–7) • Course Concept: Context-Free Grammars (CFGs), Non-regularity proofs ($0^n 1^n$), PDAs, Stack memory.

- **DeepMind Connection:** Why can't GPT or vanilla RNNs reliably evaluate deeply nested arithmetic expressions or parse deeply nested code without Chain-of-Thought? Because fixed hidden states cannot implement an unbounded LIFO stack.
- **Lecture Case Study:** Compare the failure of Vanilla RNNs on Reverse String (w^R) with the 100% success of Stack-RNNs, proving that stack memory is mathematically necessary for L_2 transduction.

Module 3: Turing Machines & The Chomsky Hierarchy (Weeks 8–10) • Course Concept: Turing Machines (TM), Linear Bounded Automata (LBA), Decidability, Non-context-free languages ($ww, a^n b^n c^n$).

- **DeepMind Connection:** The paper demonstrates that language $L = \{ww\}$ (Duplicate String) is impossible for both RNNs and Stack-RNNs, but is mastered (100%) by Tape-RNNs.
- **Lecture Case Study:** Introduce Neural Turing Machines (NTMs) and Tape-RNNs as differentiable realizations of the classical 7-tuple Turing Machine $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$.

Module 4: Computational Complexity & Modern AI Limits (Weeks 11–14) • Course Concept: Time and Space Complexity (P, NP), Circuit Complexity (TC^0, NC^1).

- **DeepMind Connection:** Why did Tape-RNN fail on Binary Multiplication? Because the model was forced to execute in linear time $O(n)$, whereas multi-digit multiplication has intrinsic computational complexity $\Omega(n \log n)$ or $\Omega(n^2)$.
- **Modern LLM Parallel:** Explain why **Chain-of-Thought (CoT)** prompting works in LLMs: CoT generates intermediate reasoning tokens, converting a constant-depth circuit (TC^0) into a dynamic-step Turing tape.

7. Recommended Student Projects & Exam Questions for CS 305

To reinforce these concepts, the following theoretical and programming assignments are designed for Level 300 students:

- 1. Pumping Lemma vs. Neural Generalization (Analytical Exercise):** *Problem:* Use the Pumping Lemma for Context-Free Languages to prove that $L = \{ww \mid w \in \{0,1\}^*\}$ is not context-free. Based on the DeepMind empirical results, explain why a Stack-RNN achieved only 52.8% test accuracy on this task, while a Tape-RNN achieved 100.0%.
- 2. The Parity Failure of Transformers (Theoretical Short Answer):** *Problem:* Parity is in L_3 (Regular) and can be decided by a 2-state DFA. Explain in terms of attention mechanisms and circuit depth why standard self-attention without recurrence struggles to compute parity over sequences of length 500 when trained only on length 40.
- 3. Chain-of-Thought as an External Tape (Discussion Topic):** *Problem:* Modern autoregressive LLMs (e.g., Gemini, GPT-4) do not have external tape heads like a Tape-RNN. However, when instructed to “think step-by-step” (Chain-of-Thought), their performance on arithmetic and code generation surges. Explain how the generated scratchpad tokens effectively function as the write-head of a Linear Bounded Automaton.

8. Conclusion

The DeepMind study by Delétang et al. firmly establishes that **the Chomsky Hierarchy is not merely a historical relic of 1950s linguistics, but the fundamental governing law of neural network length generalization.**

For students in **CS 305: Theory of Computation** at GIMPA, this research provides the ultimate justification for studying formal automata: the exact mathematical boundaries separating DFAs, PDAs, LBAs, and Turing Machines dictate where modern AI succeeds, where it fails, and how future neuro-symbolic architectures must be designed.

Monograph prepared for CS 305 (Theory of Computation) | GIMPA School of Technology | Academic Year 2026/2027